S1125886 Angelina Podoļako

Run your program a few times to get more general results.

	Statements			(%)	Branches			(%)	Total(%)			
Run	Coverage	Stone	End	Game	Coverage	Stone	End	Game	Coverage	Stone	End	Game
1	83	89	73	75	69	84	62	58	82	88	69	69
2	84	93	73	73	72	94	62	58	83	93	69	69
3	83	93	73	75	72	94	62	58	82	93	69	69
4	82	93	89	75	79	94	81	58	82	93	86	69
5	80	95	73	73	73	97	62	58	80	95	69	69
Average	**82,4**	92,6	76,2	74,2	**73**	92,6	65,8	58	**81,8**	**92,4**	**72,4**	**69**

1. Does your test generator successfully achieve a high (above 85%) coverage? Why(not)? **(10 point)**

In average I achieved the total coverage of 81,8% which is a bit lower than 85%. The thing is that for Total Coverage also TestGenerated class counts, which worsen results because of lower coverage. TestGenerated has lower coverage because if test has error message (assert) to work on, it got a lot of red lines for different , methods in those tests.

The high coverage of Stone class is because I worked a lot on trying to get maximum coverage by going from different values, methods with different probability, which changes depending on complexity of method.

The End and Game classes have lower coverage because of complexity of methods. There we needed correct object for different methods. They require consistent sequences of calls (for example, adding stones to the end, adding ends to the game), which do not always lead to the execution of all branches.

Also, maybe limit of time given for task was not enough and with not time limitations this test_generator can be extended by more complex logic of object generation with different value, and different combinations of methods with different weight.

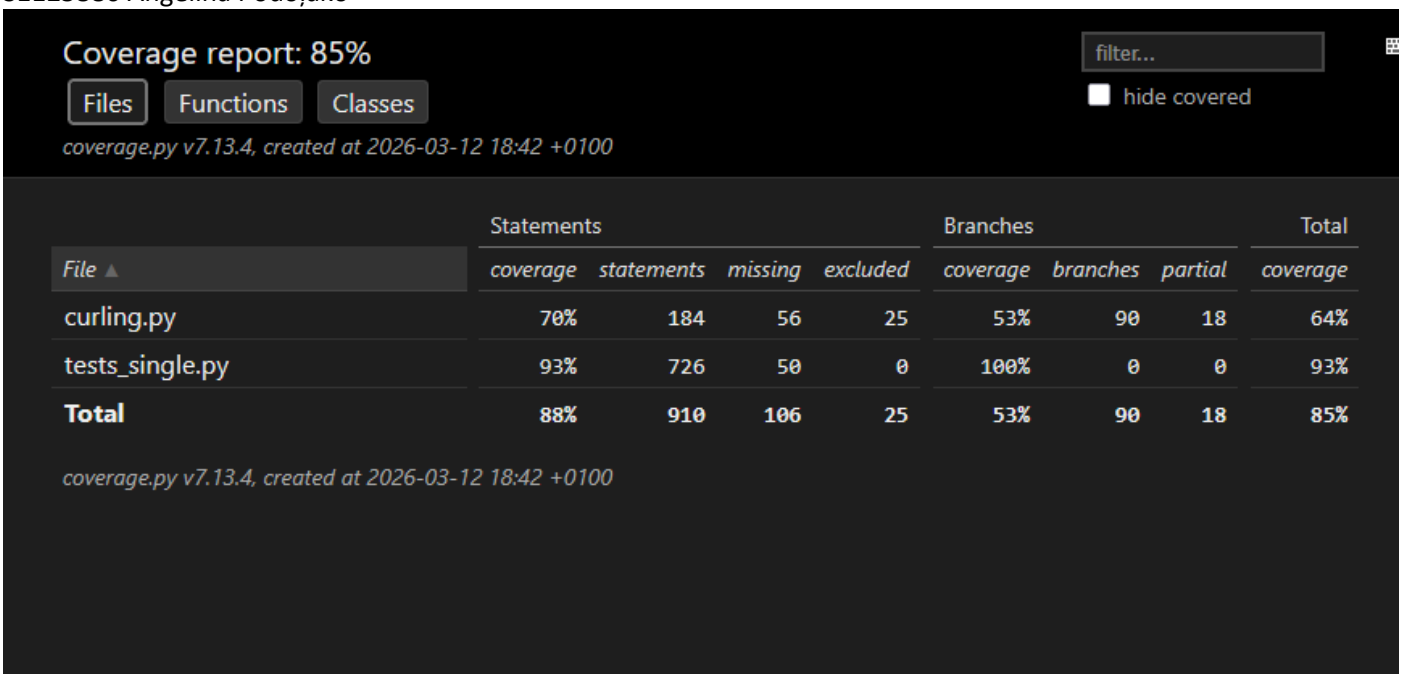
Also, I achieved around 95% of Stone class, more than 60% of End and Game classes. Firstly, coverage % were higher for each. But after changing probability for each class to probability of hidden class it dropped a bit for Stone and is higher for End class now.

2. Is your branch or your line coverage higher on average? Why? **(5 points)**

Statement average coverage (82,4%) is higher than Branches (73%). It is because Statements just checks how large part of lines of code were executed, which is simpler than to check number of executed edges(each decision choice) as in Branch coverage is done.

All branches require large variations of input values to go through more(if possible, all) edges (if-else), including bound cases and exceptions (those can we skip in line coverage).

3. Modify your test generator so that each generated sequence of calls contains only one single method, aside from the constructor. Compare its ability to achieve higher (faster) coverage with your original program. **(5 points)**



Coverage report: 85%

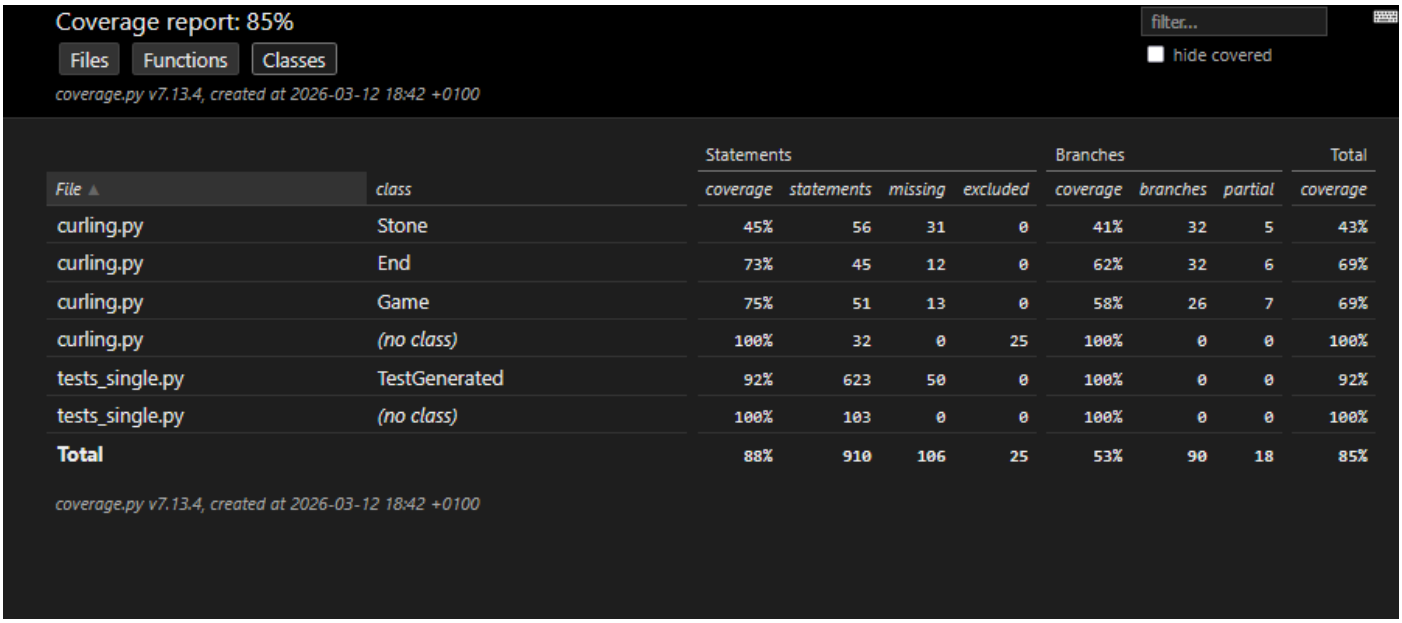
Files | Functions | Classes

filter... | hide covered

coverage.py v7.13.4, created at 2026-03-12 18:42 +0100

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
curling.py	70%	184	56	25	53%	90	18	64%
tests_single.py	93%	726	50	0	100%	0	0	93%
Total	**88%**	**910**	**106**	**25**	**53%**	**90**	**18**	**85%**

coverage.py v7.13.4, created at 2026-03-12 18:42 +0100



Coverage report: 85%

Files | Functions | Classes

filter... | hide covered

coverage.py v7.13.4, created at 2026-03-12 18:42 +0100

		Statements				Branches			Total
File ▲	class	coverage	statements	missing	excluded	coverage	branches	partial	coverage
curling.py	Stone	45%	56	31	0	41%	32	5	43%
curling.py	End	73%	45	12	0	62%	32	6	69%
curling.py	Game	75%	51	13	0	58%	26	7	69%
curling.py	*(no class)*	100%	32	0	25	100%	0	0	100%
tests_single.py	TestGenerated	92%	623	50	0	100%	0	0	92%
tests_single.py	*(no class)*	100%	103	0	0	100%	0	0	100%
Total		**88%**	**910**	**106**	**25**	**53%**	**90**	**18**	**85%**

coverage.py v7.13.4, created at 2026-03-12 18:42 +0100

	Statements			(%)	Branches			(%)	Total(%)			
Run	Coverage	Stone	End	Game	Coverage	Stone	End	Game	Coverage	Stone	End	Game
Tests	**82,4**	92,6	76,2	74,2	**73**	92,6	65,8	58	**81,8**	**92,4**	**72,4**	**69**
Tests_single	88	45	73	73	53	41	62	58	85	43	69	69
Comparison	**82,4**	-48%	-3%	-1%	**-20%**	-52%	-4%	same	**+3%**	**-49%**	**-3%**	**same**

Faster achievement of basic coverage due to the absence of consecutive lines in the test file. Not as many exceptions as in sequences of methods. That is, a higher proportion of successful tests. However, it does not test method interactions or state invariants that arise from sequential calls, and loses complex logic for different methods (e.g., move for Stone).

4. Briefly describe your strategy for generating test cases. What steps did you take to improve the performance of your test generator, and did it prove effective? Why (not)? **(10 point)**

S1125886 Angelina Podoļako

Every time I tried to extend my code to cover more of code, checked by small steps if my ideas worked (if performance increased after few test coverage checking) and after serious improvement of performance, I saved these as separate versions. There I wrote the main improvements of each version of code:
V1: Added lines of code from the assignment's example. To understand the general logic of the test generator. The structure of test generation and the principle of forming test scenarios was learn on this stage.

V2: The generation logic is implemented only for the Stone class. Added a simple check for acceptable values. The accepting and working on assertions - necessary for testing has been implemented.

V3: Added support for multiple classes(not only Stones). There is a separate function for selecting values. This made it easier to use value generation for different classes. At the same time, the logic of selecting and processing methods remained the one and same for all classes.

V4: Added a system of weights (probabilities) for selecting classes and methods. It was taken into account whether an object is required to execute the method. The logic of choosing methods is partially divided: Stone works directly with objects; End and Game perform actions on objects. Due to the more complex logic of these classes, the method selection mechanism has been changed.

V5: The logic of processing methods for different classes is completely separated. The system of weights and probabilities has been improved. Added increased probability for more complex methods to: increase test coverage, check complex scenarios more often. Added borderline cases: acceptable values, invalid values.

best: The logic of processing methods of each class has been optimized. A separate function has been introduced to check the overlap of objects (required for the End class). Added the ability to handle unknown classes. The code structure and the general logic of the generator have been improved. Various probabilities and weights were tested, after which the most appropriate values were selected. Additionally, the processing of complex methods and classes has been improved to increase test coverage.

Add a short reflection in your PDF:

1. what was the division of tasks in your group?

All tasks was done by e, as I am one member of the group.
Acknowledgments: I want to thank friend of mine for idea to make some help extra functions to use in test generator (for values, which was separated in one of the versions of code from generate_test function, and check for overlap – needed in End class to get higher coverage)

2. Did the assignment give you additional insight?

Exactly, it really helped to feel and understand how cyclic is work of test generation and how much important it is to save different versions of test generator. It was interesting, and practically showed the difference for Statements and Branch coverage.

3. Please also include information you think important during the test generator implementation.

I have different comments in code for better understanding if code will be checked as well. But maybe they are not needed.